

Sécurité des Applications

Ludovico Grabau

16.01.2022

Sécurité des Applications

Introduction

Exercise Session 1 - Building a RESTful API in Golang

Install Golang and Docker

Your first Go Application

Containerize your Go Application

Exercise Session 2 - TLS Protection

Generate your TLS keys

Setup Nginx

Setup Docker Compose

Start the container

Harden your TLS configuration

Run a TLS analyzer

Exercise Session 3 - Authentication

Basic Authentication

Implementing an OIDC Consumer using Okta

Implement Authorization

Exercise Session 4 - Secret Management

Basic Authentication Credentials

Adapting injection in Docker

Conclusion

Sources

Introduction

For the class of "Sécurité des Applications" I have worked on a project which consisted of building a RESTful API in the programming language "Go". This work got me to develop and secure an application, deployed using Docker, and accessible through endpoints for a group of students and teachers, following a desired JSON format. In this report I'll be putting forward all the steps undertaken to make this project and a few examples of the work I've completed throughout it. All the elements discussed about in this file, are accessible in the folder that comes with it containing my completed project.

Exercise Session 1 - Building a RESTful API in Golang

Developing and securing an application written in GoLang.

Install Golang and Docker

- Verify installation by installing and deploying MySQL

Your first Go Application

- Respect Data Model
- Respect expected directory structure
- Implement REST endpoints for Students
- Implement REST endpoints for Teachers

Containerize your Go Application

Build

```
> docker build --tag appsec .
```

Run

```
> sudo docker run -p 8080:8080 appsec
```

Exercise Session 2 - TLS Protection

Protect the REST endpoints by implementing a reverse proxy.

Generate your TLS keys

```
> mkdir certs
> openssl req -x509 -newkey rsa:4096 -keyout certs/key.pem -out certs/cert.pem
-days 365 -nodes
```

Setup Nginx

- Install
- Configure
- Reverse Proxy your application's HTTP traffic

```
<your_subdomain.com> => server_name localhost;
<your_ssl_cert.pem> => ssl_certificate /certs/cert.pem;
<your_ssl_private_key.key> => ssl_certificate_key /certs/key.pem;
```

Setup Docker Compose

```
version: '3'
services:
  nginx:
    image: nginx:latest
    container_name: nginx
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./certs:/certs
      - ./logs:/etc/nginx/logs/
    ports:
      - 80:80
      - 443:443
  appsec:
    image: appsec:latest
    container_name: appsec
    expose:
      - 8080:8080
```

Start the container

```
> sudo docker-compose up
```

Harden your TLS configuration

List 5 parameters that you could include to your configuration to improve it :

1. Set the number of worker processes equal to the number of processors,
2. Enable keep-alive connections,
3. Enable the shared session cache,
4. Disable the built-in session cache,
5. And possibly increase the session lifetime (by default, 5 minutes).

Run a TLS analyzer

List what can be improved further! :

- Make sure that my security certificate doesn't expire soon and be able to update it if needed.
- Check if I'm using outdated protocols and fix it by ensuring to use latest versions.
- Ensure to use updated versions of encryption algorithms.
- Use an HTTPS encryption.

Exercise Session 3 - Authentication

Implement authentication and authorization on the REST endpoints by implementing basic authentication and Oauth 2.0/OIDC on them.

Basic Authentication

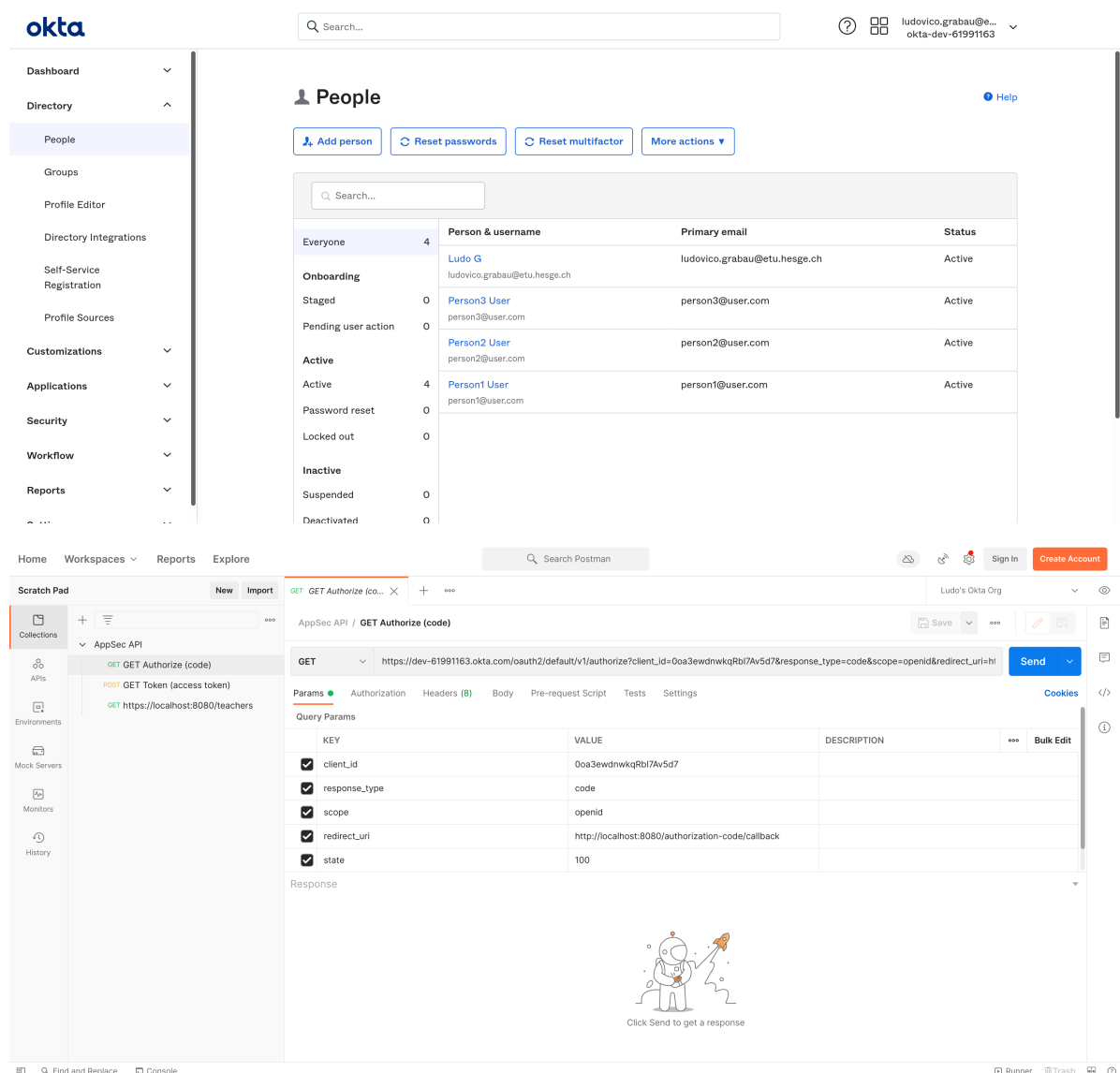
Username	Password
foo	bar
aristote	Eucl1de

Adding Authorization Logic :

```
- func checkAuthorizations(c *gin.Context) { ... }  
- authorized.GET("/students", getStudents) -> authorized.GET("/students",  
checkAuthorizations, getStudents)
```

Implementing an OIDC Consumer using Okta

- Getting Started with Okta
- Creating an Application Integration
- validate the JWT token in Go
- Run your example with Postman
- Implement authorization



The image displays two web interfaces side-by-side. The top interface is the Okta admin console, showing the 'People' section with a list of users. The bottom interface is the Postman API client, showing a GET request to the Okta OAuth2 authorization endpoint with query parameters for client_id, response_type, scope, redirect_uri, and state.

Okta Admin Console - People Section

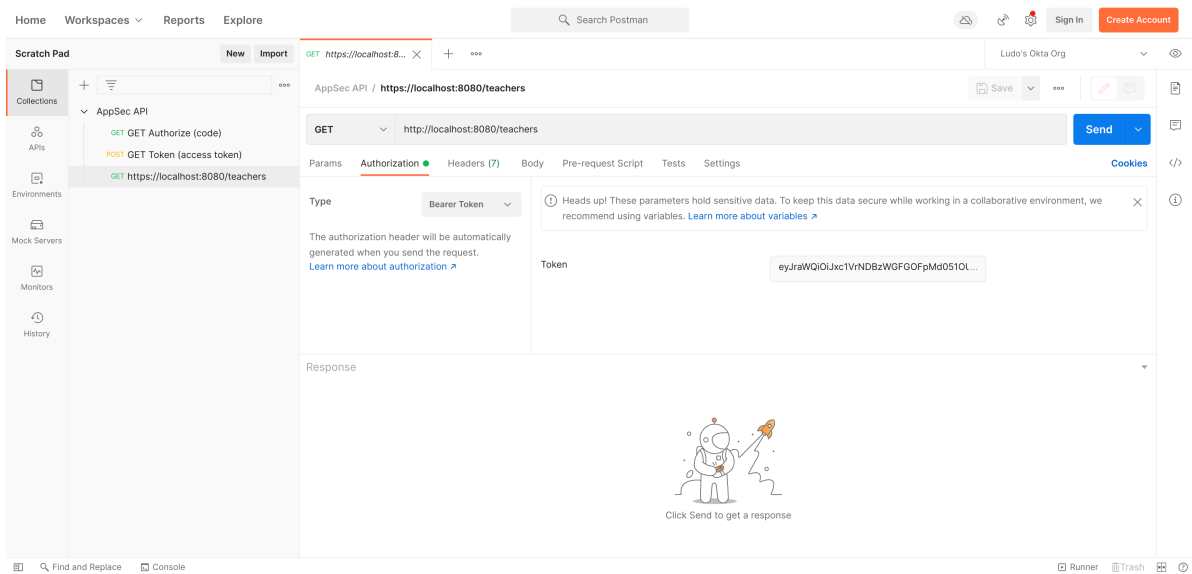
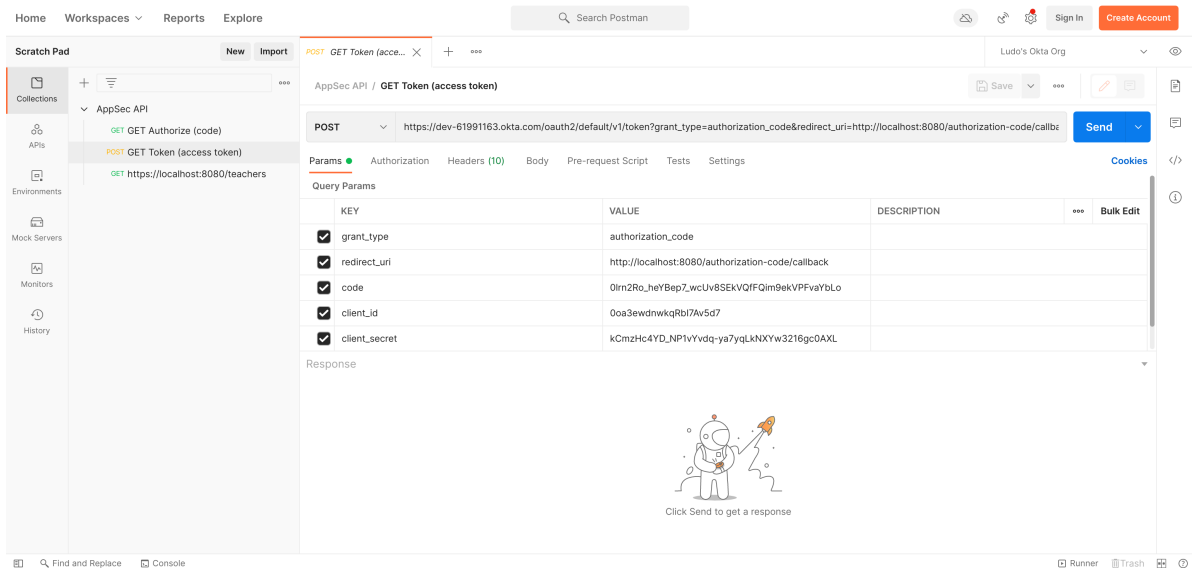
Category	Count	Person & username	Primary email	Status
Everyone	4	Ludo G ludovico.grabau@etu.hesge.ch	ludovico.grabau@etu.hesge.ch	Active
Onboarding	0	Person3 User person3@user.com	person3@user.com	Active
Staged	0	Person2 User person2@user.com	person2@user.com	Active
Pending user action	0	Person1 User person1@user.com	person1@user.com	Active
Active	4			
Active	4	Person1 User person1@user.com	person1@user.com	Active
Password reset	0			
Locked out	0			
Inactive	0			
Suspended	0			
Deactivated	0			

Postman API Client - GET Request

URL: `https://dev-61991163.okta.com/oauth2/default/v1/authorize?client_id=00a3ewdnwqRbi7Av5d7&response_type=code&scope=openid&redirect_uri=http://localhost:8080/authorization-code/callback`

Query Params:

KEY	VALUE	DESCRIPTION
client_id	00a3ewdnwqRbi7Av5d7	
response_type	code	
scope	openid	
redirect_uri	http://localhost:8080/authorization-code/callback	
state	100	



Implement Authorization

Added & Modified :

```
- var toValidate = map[string]string { ... }
- func verify(c *gin.Context) bool { ... }
```

Created : `func checkOktaAuthorizations(c *gin.Context) { ... }`

```
Modified : oktaAuthorized.GET("/teachers", getTeachers) ->
           oktaAuthorized.GET("/teachers", checkOktaAuthorizations,
           getTeachers)
```

Exercise Session 4 - Secret Management

Resolving the issue of secrets lying in code by injecting all secrets using environment variables.

Basic Authentication Credentials

```
var accounts = make(map[string]string)

var login string
var password string

login = os.Getenv("LOGINS")
password = os.Getenv("PASSWORDS")

logins := strings.Fields(login)
passwords := strings.Fields(password)

for k, v := range logins {
    accounts[v] = passwords[k]
}

authorized := router.Group("/").Use(gin.BasicAuth(accounts))
```

Adapting injection in Docker

```
...
appsec:
    ...
    env_file: appsec/.env
    ...
```

Conclusion

In conclusion, we've successfully created the main.go that holds all of our program, a Dockerfile which allows us to specify the instructions to build our Docker image on it, added and configured a nginx.conf file to reverse proxy our application's HTTP traffic, setup a docker-compose.yml to define the behavior of our docker to run the application, a cert.pem and a key.pem for our TLS endpoints, added authentication and authorization using Okta which was connected to Postman to test it and finally passed our credentials to an .env file to not hold confidential data in our code and still be able to use it for authentication.

I've worked on this project alone, however I occasionally recieved help from Dylan Perry and Scott Birner.

Every point that was mandatory in the making of this work has been done. Which means that all the steps stated in this report that lead to the creation of my RESTful API, have been completed.

Sources

<https://golang.org/doc/install>

<https://docs.docker.com/get-docker/>

<https://docs.docker.com/compose/install/>

<https://github.com/gin-gonic/gin>

<https://golang.org/doc/tutorial/web-service-gin>

<https://docs.docker.com/language/golang/>

<https://www.digicert.com/kb/ssl-support/openssl-quick-reference-guide.htm>

https://nginx.org/en/docs/http/nginx_ssl_module.html#ssl_ciphers

<https://github.com/tls-attacker/TLS-Scanner>

<https://developer.okta.com>

<https://developer.okta.com/blog/2021/02/17/building-and-securing-a-go-and-gin-web-application#how-to-validate-an-access-token-in-go>

<https://github.com/oktadev/okta-go-gin-vue-example>

<https://www.postman.com/downloads/>

<https://developer.okta.com/code/rest/>

<https://gobyexample.com/environment-variables>

<https://docs.docker.com/compose/environment-variables/#pass-environment-variables-to-containers>